Pathfinder

Interactive Pathfinding Visualizer

C Programming Mini Project

Project Team & Details

Team Members

Rashi Malpani	1262252822
Akshara Dubey	1262253416
Sharvani Walawalkar	1262251167
Devesh Patil	1262251847
Neer Agrawal	1262252059

Project Information

Subject
Foundation of Programming

Subject Teacher
Dr. Uma Pujeri

Academic Session
2025-26

Programming Language
C Language

Problem Statement

The Challenge

Pathfinding algorithms are fundamental to computer science, yet understanding how they work internally can be abstract and difficult for students. Traditional learning methods rely on static diagrams and theoretical explanations, which fail to convey the **dynamic, step-by-step nature** of algorithm execution.

There is a need for an interactive, visual tool that allows users to see algorithms in action, experiment with different scenarios, and gain intuitive understanding through hands-on exploration.

Our Solution

We developed **Pathfinder**, an interactive desktop application built in C that visualizes the A* pathfinding algorithm in real-time. Users can create custom grids, place obstacles, set start and end points, and watch the algorithm find the optimal path step by step.

Project Objectives

01

Implement A* Algorithm

Develop a complete, working implementation of the A* pathfinding algorithm in C, including proper data structures for open/closed lists and heuristic calculations.

02

Create Interactive GUI

Build a user-friendly graphical interface using SDL2 that allows users to interact with the grid, place nodes, and control the visualization.

03

Enable Maze Generation

Implement a recursive backtracking algorithm to generate random mazes, providing users with complex scenarios to test the pathfinder.

04

Real-Time Visualization

Provide animated, step-by-step visualization of the algorithm's execution, showing how nodes are explored and the path is constructed.

05

User Control Features

Allow users to place start/end points, draw walls, clear paths, reset the grid, and trigger pathfinding with intuitive controls.

Primary Goal: Build a functional, educational, and visually appealing pathfinding visualization tool that demonstrates core computer science concepts.

Libraries & Tools Used

SDL2

Simple DirectMedia Layer

Cross-platform development library for graphics, window management, and event handling. Provides low-level access to audio, keyboard, mouse, and display.

Graphics Rendering | Event Handling | Window Management

SDL2_ttf

TrueType Font Library

Extension library for SDL2 that enables rendering TrueType fonts. Used for displaying button labels, path length, and UI text elements.

Text Rendering | Font Management | UI Labels

Standard C Libraries

stdio.h	Input/output operations
stdlib.h	Memory allocation, random numbers
string.h	String manipulation
stdbool.h	Boolean data type
math.h	Mathematical functions

Development Tools

Compiler	GCC / MinGW
IDE	Code: VS Code
Debugger	GDB

Methodology - Project Architecture

Modular Header Structure

Node.h

Node Structure Definition

Defines the Node struct with position, costs (g, h, f), parent pointer, and state. Includes color helper functions for rendering.

State.h

State Enumeration

Defines all possible node states: EMPTY, WALL, START, END, OPEN, CLOSED, PATH as an enum for type safety.

Grid.h

Grid Management

Manages the 2D grid of nodes, handles user interactions, maze generation, pathfinding execution, and rendering.

AStar.h

A* Algorithm Implementation

Core pathfinding logic with open/closed lists, neighbor exploration, cost calculations, and path reconstruction.

Main.h

Application Entry Point

SDL2 initialization, main event loop, UI rendering, button handling, and application state management.

Architecture Layers

- **UI Layer**
  Buttons, labels, event handling
- **Grid Layer**
  Node management, rendering
- **Algorithm Layer**
  A* logic, maze generation
- **Data Layer**
  Node structures, state enum

Design Principles

- **Modularity:** Each header handles specific functionality
- **Reusability:** Components can be easily extended
- **Maintainability:** Clear separation of concerns

Methodology - A* Algorithm Logic

Cost Calculations

g — gCost – Distance from Start

Actual path cost from the starting node to the current node. Increments by 1 for each step.

h — hCost – Heuristic to End

Estimated cost from current node to target usingeucledian distance

f — fCost – Total Cost

Sum of gCost and hCost: f = g + h. Node with lowest fCost is explored next.

Algorithm Steps

1. Add start node to open list
2. Find node with lowest fCost in open list
3. Move current node from open to closed list
4. For each neighbor, calculate costs and update parent
5. Repeat until end node is reached or open list is empty
6. Reconstruct path by following parent pointers

Key Code Snippet

```
// AStar.h - Core pathfinding loop
while
    (openSize > 0) {
        Node* current     = getLowestF();
        if    (current == end) {         // Path found!
            pathFound =   true    ;   break   ;
        }
        removeFromOpen(current);
        addToClosed(current);
        for    (   each neighbor     ) {
            if    (neighbor is wall         or   in closed)
                continue    ;
            neighbor->gCost = current->gCost + 1;
            neighbor->hCost = heuristic(neighbor, end);
            neighbor->fCost = gCost + hCost;
            neighbor->parent = current;
            addToOpen(neighbor);
        }
    }
```

Why A*?

- ✓ **Optimal:** Finds shortest path when heuristic is admissible
- ✓ **Efficient:** Uses heuristic to guide search toward goal
- ✓ **Complete:** Guaranteed to find a path if one exists
- ✓ **Versatile:** Works with any admissible heuristic function

Methodology - Maze Generation

Recursive Backtracking Algorithm

We implement a **randomized depth-first search** algorithm that carves passages through a grid, creating perfect mazes (every point is reachable, no loops, no inaccessible areas).

1 Initialize Grid

Fill entire grid with walls (1s). Choose random starting cell with odd coordinates.

2 Carve Passages

From current cell, randomly select unvisited neighbor 2 cells away. Remove wall between them.

3 Stack-Based Tracking

Push current cell to stack. Move to neighbor. If no unvisited neighbors, pop from stack (backtrack).

4 Completion

Repeat until stack is empty. Result: perfect maze with exactly one solution.

Implementation Details

```
// Grid.h - Maze generation
void
    grid_generate_maze
    (Grid* g) {
        // Fill with walls
        for  (  all cells      ) maze[i][j] = 1;
        // Random start (odd coordinates)
        int   row = rand() % width;
        while   (row % 2 == 0) row = rand();
        maze[row][col] = 0;        // Mark as passage
        // Recursive carving
        maze_recursion(g, row, col);
        // Transfer to grid
        for  (  all cells      )
            if   (maze[i][j] == 1)
                grid_get(g,i,j)->state = STATE_WALL;
    }
```

Algorithm Features

- **Perfect Maze:** Exactly one path between any two points
- **Randomized:** Different maze every time
- **Efficient:** O(N) time complexity where N = number of cells
- **Stack-Based:** Iterative implementation avoids recursion limits

Methodology - UI & Interaction

Action Buttons

Visualize Path
Starts A* algorithm and animates the search process

Generate Maze
Creates a random maze using recursive backtracking

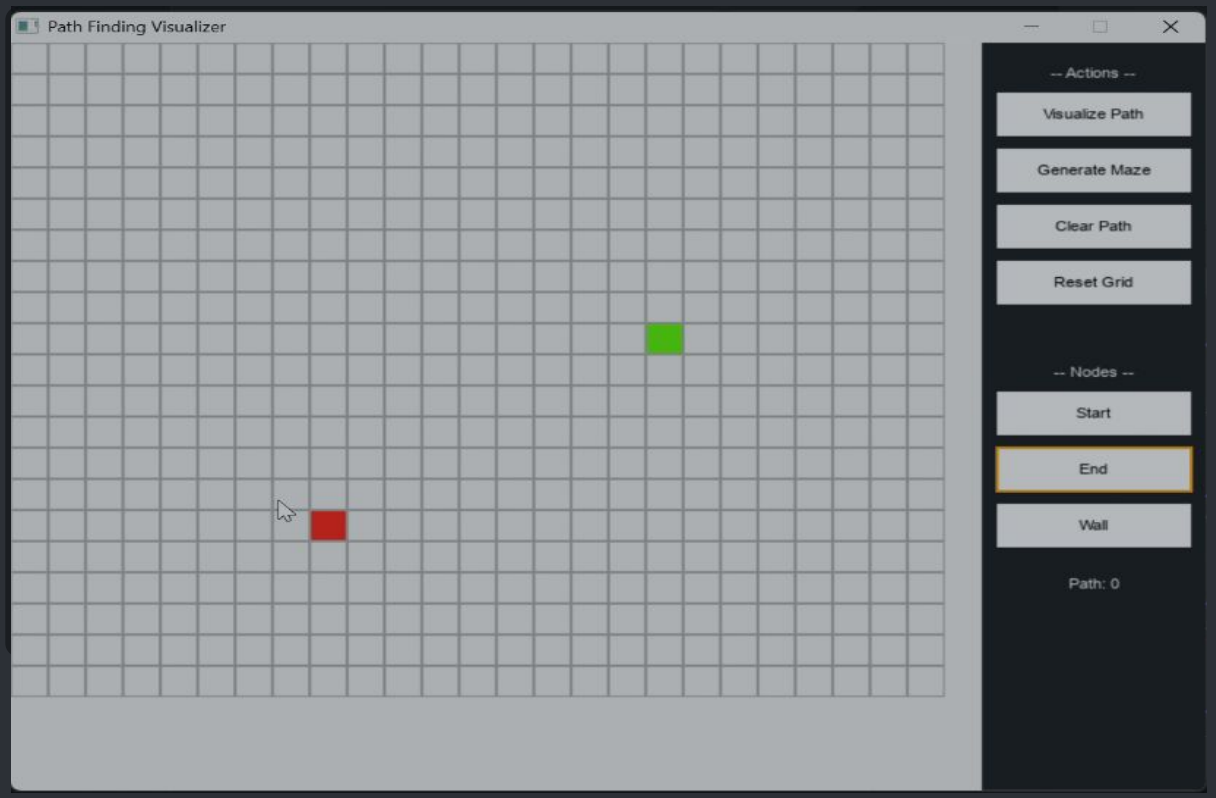
Clear Path
Removes visualization but keeps walls and start/end

Reset Grid
Clears everything and restores default start/end

Node Selection

Start	End	Wall
Green node	Red node	Dark node

Window Layout



Interaction Methods

- **Click:** Place nodes or trigger actions
- **Drag:** Draw multiple walls quickly
- **Hover:** Button highlighting effects
- **ESC Key:** Exit application

Grid Setup

Initial Grid Layout

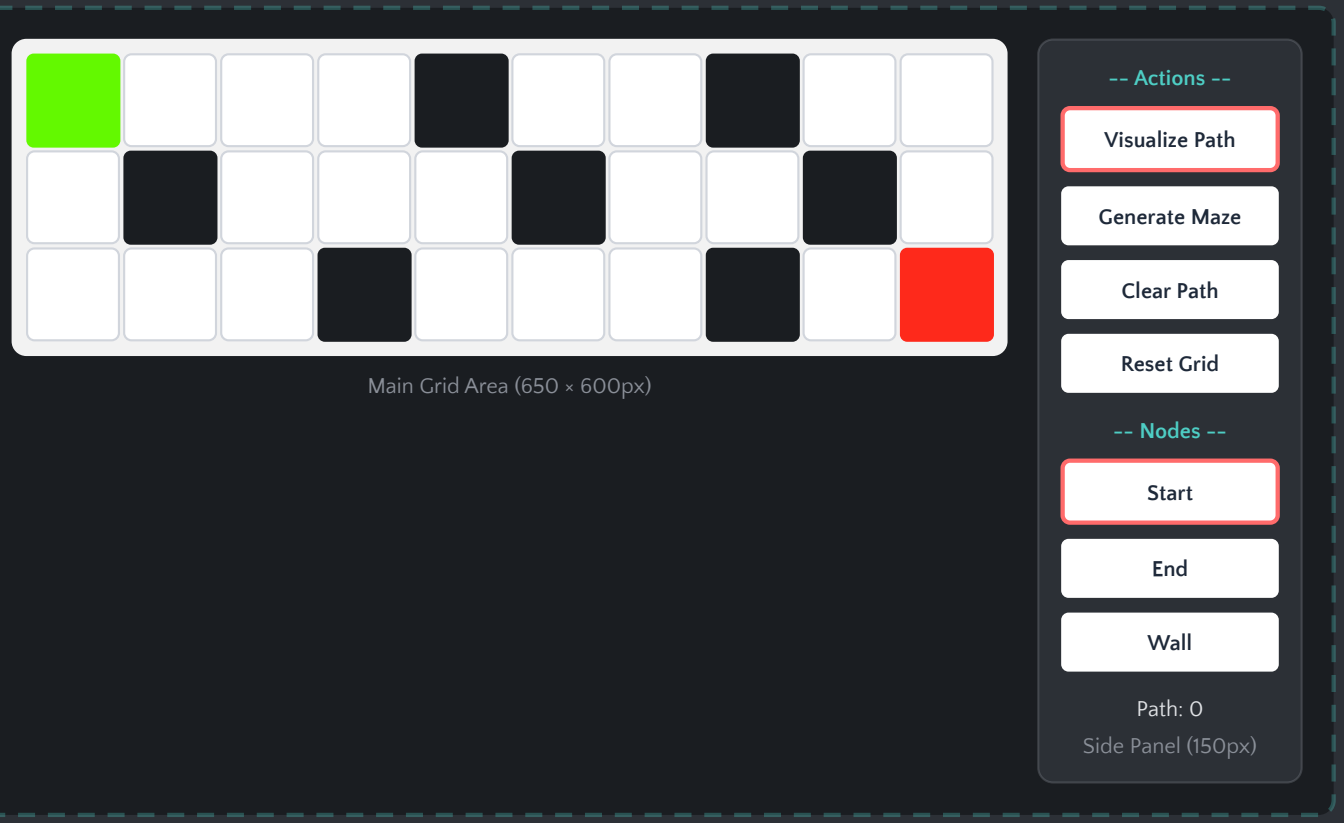


Figure 1: Application interface showing the initial grid with start node (green), end node (red), user-placed walls (dark), and the control panel on the right side.

Interface Elements

Start Node
Green - Pathfinding origin

End Node
Red - Target destination

Wall Nodes
Dark - Impassable obstacles

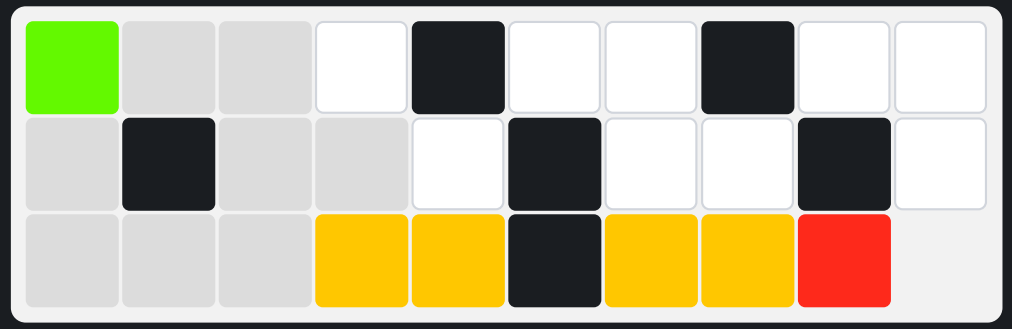
Empty Cells
White - Walkable areas

Key Features

- ✓ Interactive grid with 25×25 pixel cells
- ✓ Side panel with action buttons
- ✓ Node type selection (Start/End/Wall)
- ✓ Path length display
- ✓ Hover effects on buttons

Path Visualization

A* Algorithm in Action



-- Actions --

Visualize Path

Generate Maze

Clear Path

Reset Grid

-- Nodes --

Start

End

Wall

Path: 5

Controls Disabled

Figure 2: A* algorithm visualization showing explored nodes (gray), the optimal path (yellow), walls (dark), start (green), and end (red). The algorithm efficiently finds the shortest path around obstacles.

Visualization States

CLOSED Nodes
Gray - Already explored

PATH Nodes
Yellow - Final optimal path

OPEN Nodes
Blue - Candidates to explore

Animation Process

1. Algorithm starts from green node
2. Explores neighbors (blue nodes)
3. Marks explored nodes (gray)
4. Reaches red node - path found!
5. Highlights optimal path (yellow)

Note: Controls are disabled during visualization to prevent interference.

Key Features & Highlights

Real-Time Visualization

Watch the A* algorithm explore the grid step-by-step with smooth animations. See how nodes are evaluated, added to open/closed lists, and how the optimal path emerges.

Animated | Step-by-step | Educational

Interactive Grid Editing

Click and drag to place walls, move start/end points anywhere on the grid. Create custom scenarios to test different pathfinding situations and edge cases.

Click & Drag | Customizable | Flexible

Random Maze Generation

Generate perfect mazes instantly using recursive backtracking algorithm. Each maze is unique with exactly one solution, providing endless challenge scenarios.

Random | Perfect Maze | Unique

Optimal Pathfinding

A* algorithm guarantees finding the shortest path when one exists. Uses intelligent heuristic to prioritize exploration toward the target, making it efficient.

Optimal | Efficient | Guaranteed

Intuitive User Interface

Clean, modern UI with clear visual feedback. Side panel provides easy access to all controls. Color-coded nodes make it easy to understand the grid state at a glance.

Clean | Modern | User-Friendly

Clean C Implementation

Well-structured, modular C code with proper memory management. Header-based organization makes the codebase easy to understand, extend, and maintain.

Modular | Clean Code | Extensible

Project Impact: Successfully demonstrates core computer science concepts while providing an engaging, interactive learning tool for pathfinding algorithms.

Conclusion

What We Achieved

Fully Functional Application

Successfully built a complete, working pathfinding visualizer in C with SDL2 that demonstrates the A* algorithm in real-time.

Educational Value

Created an interactive tool that helps students and learners understand how pathfinding algorithms work through hands-on experimentation.

Feature-Rich Implementation

Delivered all planned features: A* visualization, maze generation, interactive grid editing, and intuitive user interface.

Future Enhancements

- Add more algorithms (Dijkstra, BFS, DFS)
- Support diagonal movement
- Add weighted nodes with different costs
- Export/import grid configurations
- Speed control for visualization

Learning Outcomes

- **Algorithm Understanding:** Deep knowledge of A* pathfinding, heuristics, and search algorithms
- **C Programming:** Advanced skills in memory management, pointers, and data structures
- **Graphics Programming:** Experience with SDL2 for creating graphical applications
- **Software Engineering:** Modular design, code organization, and debugging techniques
- **Problem Solving:** Overcoming implementation challenges and optimization

> **Reflection:** This project successfully bridged the gap between theoretical algorithm knowledge and practical implementation, creating a valuable educational tool while strengthening our programming skills.

Thank You